

Diffusion Autoencoders (DAE) no Espaço Latente

Geração Condicionada e Classificação a partir de Latentes

Prof. Anderson Rocha — Visão Computacional

24 de novembro de 2025

- ➊ Motivação: por que difundir em latentes (z) e não em pixels
- ➋ DAE/LDM: revisão rápida e papel da UNet latente
- ➌ Condicionamento por texto/imagem e CFG
- ➍ Amostragem (DDPM, DDIM, DPM) e ganhos de tempo em z
- ➎ **Latentes para Classificação**
 - (A) *Linear probe* em z_0 (VAE/AE)
 - (B) *Denoising features* ao longo de t
 - (C) Classificador no ruído (ε) e em $\hat{z}_0$
 - (D) *Classifier Guidance* e *Classifier-Free Guidance* para rótulos
 - (E) **Transformer Diffusion (DiT)** para classificação
- ➏ Demonstrações conceituais + check-list experimental
- ➐ Perguntas, ablações e conclusões

Por que difundir no espaço latente?

Custo computacional:

- Em pixels: custo $\mathcal{O}(HW)$ (atenção $\mathcal{O}((HW)^2)$).
- Autoencoder (fator de compressão f): $z \in \mathbb{R}^{(H/f) \times (W/f) \times c}$.
- Difusão em z : menos passos (10–30), atenção mais barata, *speedup* grande.

Pipeline geral:

$$E_\phi : x \mapsto z_0, \quad \text{difusão em } z_t, \quad D_\psi : z_0 \mapsto \hat{x}$$

Trade-off: compressão excessiva $\Rightarrow$ perda de detalhe, artefatos do decodificador.

DAE/LDM: o que cada parte faz

Autoencoder (geralmente VAE):

$$z_0 = E_\phi(x), \quad \hat{x} = D_\psi(z_0), \quad \mathcal{L}_{\text{rec}} + \lambda \text{KL}(q_\phi(z|x) \parallel \mathcal{N}(0, I))$$

Difusão no latente (UNet):

$$q(z_t|z_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} z_{t-1}, \beta_t I), \quad z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Treino (predizer ruído):

$$\min_{\theta} \mathbb{E}_{z_0, \varepsilon, t} [\|\varepsilon - \varepsilon_\theta(z_t, t, c)\|_2^2]$$

Ponto-chave: A UNet latente NÃO “gera dados”. Ela prediz o ruído (ou v). A imagem vem de $D_\psi(\hat{z}_0)$.

Passo reverso genérico (exemplo):

$$z_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(z_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\varepsilon} \right) + \sigma_t \eta, \quad \eta \sim \mathcal{N}(0, I)$$

Recuperar z_0 de $\hat{\varepsilon}$:

$$\hat{z}_0 = \frac{z_t - \sqrt{1 - \bar{\alpha}_t} \hat{\varepsilon}}{\sqrt{\bar{\alpha}_t}}$$
$$\hat{x} = D_\psi(\hat{z}_0)$$

Intuição: a UNet fornece a **direção de denoising**; o decodificador reconstrói o domínio de pixels.

Condicionalismo com texto (prompts) via Cross-Attention

Texto $\rightarrow$ **embeddings**: tokenizer $\Rightarrow$ encoder de texto (CLIP/T5) $\Rightarrow c = \{e_1, \dots, e_L\}$.

Cross-attention no UNet:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad Q = \text{features do UNet}, K, V = \text{texto } c$$

Classifier-Free Guidance (CFG):

$$\hat{\varepsilon} = \varepsilon_\theta(z_t, t, \emptyset) + s(\varepsilon_\theta(z_t, t, c) - \varepsilon_\theta(z_t, t, \emptyset))$$

s : escala de guidance (típico 3–9).

Schedulers e amostragem eficiente em latentes

- **DDPM**: estocástico, estável, mais passos.
- **DDIM**: determinístico ($\sigma_t = 0$), menos passos.
- **DPM-family / ODE-SDE**: 10–30 passos com boa qualidade.

Pseudocódigo (geração):

```
c = TextEncoder(tokenize(p))  
z = randn(latent_shape)  
for t = T..1:  
    eps_u = UNet(z,t, c=None)  
    eps_c = UNet(z,t, c=c)  
    eps = eps_u + s*(eps_c - eps_u)  
    z = scheduler_step(z,t,eps)  
x = Decoder(z)
```

Objetivo: usar o espaço latente para classificação

Queremos **mapear dados** para **representações** úteis e **classificar**:

$$x \xrightarrow{E_\phi} z_0 \in \mathbb{R}^{h \times w \times c} \Rightarrow \text{features} \Rightarrow \hat{y}$$

Fontes de features:

- 1 **Latente do VAE/AE** (z_0 “limpo”).
- 2 **Denoising features** do UNet ao longo de t .
- 3 **Estimativas** $\hat{\epsilon}(z_t, t, \cdot)$ e $\hat{z}_0$.

(A) *Linear probe* em z_0 (simples e eficaz)

Passos práticos:

- 1 Treine E_ϕ, D_ψ (VAE/AE) até boa reconstrução; congele.
- 2 Para cada x , compute $z_0 = E_\phi(x)$.
- 3 Agregue z_0 (global average pooling ou flatten + MLP).
- 4 Treine um **classificador linear** em cima dessas features.

Vantagens: baixo custo, boa linha de base; útil em *few-shot*.

Dica: se o VAE estiver fraco, use perda perceptual (LPIPS) e ajuste KL.

(B) *Denoising features* ao longo do tempo t

Ideia: extrair features internas da UNet enquanto ela denoisa z_t .

Pipeline:

- 1 Amostre $t \sim \mathcal{U}\{1, \dots, T\}$, gere $z_t = \sqrt{\bar{\alpha}_t}z_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon$.
- 2 Passe z_t pela UNet (com ou sem condição c).
- 3 Colete **features intermediárias** (blocos médio/alto).
- 4 Faça pooling + MLP para classificar.

Variações: concatenar múltiplos t ; usar $\hat{z}_0$ estimado como *feature*.

(C) Classificar via $\hat{\varepsilon}$ e $\hat{z}_0$

Por que funciona? $\hat{\varepsilon}$ codifica a estrutura a ser removida.

Estratégias:

- Head de classificação sobre tensores usados para prever $\hat{\varepsilon}$ (freeze o resto).
- MLP/linear sobre $\hat{z}_0$ estimado.
- *Ensembling* sobre vários t (média de logits).

(D) Guidance por rótulos: *Classifier* & *Classifier-Free*

Classifier Guidance (classificador explícito):

$\nabla_{z_t} \log p_\varphi(y | z_t) \Rightarrow$ ajusta o passo de denoising em direção à classe y

Classifier-Free para rótulos (sem classificador externo):

$$\hat{\epsilon}_y = \epsilon_\theta(z_t, t, \emptyset) + s(\epsilon_\theta(z_t, t, y) - \epsilon_\theta(z_t, t, \emptyset))$$

Treino: % de *drop* de condição para aprender caminhos cond/unc.

(E) Transformer Diffusion (DiT): visão geral

Troca UNet por Transformer operando em **patches latentes**:

- $z_t \Rightarrow \textit{patchify} \Rightarrow$ sequência.
- Embeddings de tempo t somados/injetados.
- Cross-attention com texto/condição (se houver).
- Saída: predição de ε (ou v).

Para classificação:

- Adicionar token [CLS] e uma cabeça linear para K classes.
- Fine-tuning leve: congelar base e treinar cabeça (e últimas L camadas, se preciso).
- Alternativa: pooling sobre tokens (mean/median) + MLP.

DiT: passo a passo para classificação (prático)

Treino em dois estágios:

- 1 **Pré-treino** DiT como *denoiser* no latente (tarefa padrão de difusão).
- 2 **Fine-tuning leve:**
 - Congele a base do DiT.
 - Insira [CLS] e uma cabeça linear para K classes.
 - Treine só a cabeça (e, se necessário, as últimas L camadas).

Inferência:

- Extraia features do [CLS] (ou pooled) via $z_0 = E_\phi(x)$ ou alguns z_t .
- Gere $\hat{y}$ pela cabeça linear.

Comparar prompts e CFG:

- Prompt básico vs. enriquecido (estilo/composição).
- Escala $s \in \{1, 4, 8, 12\}$: fidelidade vs. diversidade.

Latente para classificação:

- *Linear probe* em z_0 vs. *denoising features*.
- Ensembling sobre múltiplos t .

DiT: diagrama mental de *patchify* + [CLS] + cabeça linear.

Check-list experimental (recomendado)

Setup:

- VAE/AE com perda perceptual (LPIPS) e KL balanceado.
- Difusão no latente com ε -prediction (ou v -prediction).
- Precisão mista (bf16/fp16), gradient accumulation.

Ablations:

- Compressão $f \in \{4, 8\}$ e canais c .
- Schedulers: DDIM vs DPM-Solver (10/20/30 passos).
- *Linear probe* vs denoising features vs $\hat{z}_0$ estimado.
- DiT: só cabeça vs cabeça+últimas L camadas.

Métricas: Acurácia/ROC-AUC (class.), FID/CLIP-score (geração), tempo/ imagem.

- **UNet/DiT latentes não “geram” diretamente:** predizem ε (ou v); a imagem é $D_\psi(\hat{z}_0)$.
- Difusão em $z \Rightarrow$ **rápida e eficiente**; condicionamento via **cross-attention + CFG**.
- **Classificação:**
 - (A) *Linear probe* em z_0 .
 - (B) *Denoising features* multi- t do UNet/DiT.
 - (C) Cabeça sobre $\hat{\varepsilon}/\hat{z}_0$.
 - (D) Guidance por rótulos (explícito ou *free*).
 - (E) **DiT**: patches + [CLS] + cabeça linear.